

Programming Tools for Scientific Computing: Project assignment



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Prof. Dr.-Ing. Christian Hasse, Driss Kaddar M.Sc., Pascal Steffens M.Sc.

Summer Semester 2024 (2023/07/21)

Subject of the final project of the Tutorial Programming Tools for Scientific Computing is the implementation of a finite difference method for the solution of the two-dimensional transient heat equation. A bash script will be used to start the simulation with different input parameters and do a parameter variation. To test the bash script and the solver, you will set up a docker container and a CICD pipeline. The tasks cover most of the topics that we have touched on in the lecture and will be graded separately.

Contents

1	Tasks	1
1.1	Solver implementation in C++	1
1.2	Build process with CMake	2
1.3	Parameter variation script with Bash	3
1.4	Container environment in Docker	3
1.5	CI/CD with Git	3
1.6	README.md	4
2	Scientific and algorithmic description	4
2.1	Problem description	4
2.2	Spatial and temporal discretization	4
2.3	Algorithmic solution	6
3	Final report	6
3.1	Content	6
3.2	Form	6

1 Tasks

1.1 Solver implementation in C++

1. Implementation of the main program
 - a) Write an executable program that runs your heat equation simulation. This program should follow the [scientific and algorithmic description](#) below. It should:
 - i. Implement a Newton method to iteratively solve the non-linear system of equations. See "Solution of non-linear equations".
 - ii. Contain an output function to write the results at specified time intervals. See "Writing output".

-
- iii. Write the current time and the number of iterations of the Newton method to stdout. See "Writing output".
 - iv. Accept the following command line arguments: Number of grid points in x and y direction, number of time steps, output files write interval, radiation coefficient beta, frequency coefficient gamma.
 - v. Consider storing case-specific information (e.g., number of grid points, current Jacobian matrix, ...) in a struct or class that you pass to the functions to limit the amount of function parameters and avoid too many global variables.
- b) Solution of the linear equations
- i. Download and build the [Eigen library](#).
 - ii. Write a function that solves the equation system with the linear source term using an appropriate function from Eigen. For simplicity, it is advised to use a dense matrix format.
- c) Solution of non-linear equations
- i. Write a function that implements a Newton(-Rhapson) method for the solution of the non-linear algebraic equations arising from the time discretization with the backward Euler method. It is advised to follow the algorithmic description below.
 - ii. Use an Eigen library function to solve the system of linear equations arising in the Newton method.
 - iii. Add the ability to switch between a linear solution function and a non-linear solution function depending on the β coefficient (see below).
- d) Writing output
- i. Download and build the [VTK library](#)¹.
 - ii. Write a function that saves the data of the current time step in a VTK file.
 - iii. Write a function that compares the solution of the current time step with the analytic solution and returns the L2 error norm.

1.2 Build process with CMake

Write a CMakeLists.txt file that configures your project. It should:

1. Look for the Eigen and the VTK library. You can assume that both libraries are already installed and they don't have to be installed as part of your build process.
2. Compile all source files.
3. Structure your project in libraries and configure CMake accordingly.
4. Install the binaries (and header files if needed) to the directory specified by CMAKE_INSTALL_PREFIX.

¹Visualization Toolkit

1.3 Parameter variation script with Bash

Write a bash script to run a parameter variation on the possible [command line arguments](#) (e.g., time step, coefficients,...). The bash script should:

1. Make use of functions.
2. Accept user input through command line arguments (parameter to vary + values and parameter variation name). Set defaults for values that are not explicitly provided.
3. Write a start file for a single simulation run (to be able to test the bash script and document the input parameters of your simulation), e.g.,

```
#!/bin/bash
./solver 100 10 10 1 1
# number of grid points in each direction (n),
# number of timesteps (Nt), write interval in time steps,
# radiation source term (beta),
# frequency coefficient (gamma)
```

4. Run the start file.
5. Move start file and results in a separate folder.
6. Repeat for next value.
7. Parameter variation 1: Set $\beta = 0$, $n = 16$ and vary γ between 0.0, 0.01, 0.1, 1.0. For each value of γ , vary the number of timesteps between 1, 10, 100 and compute the L2 error (between your and the analytical solution) at the final timestep. Include a plot of L2 against time step size for the four values of γ in your report.
8. Parameter variation 2: Set $\gamma = 1$, $n = 16$ and $N_t = 10$. Vary β between 0.0, 0.1, 1.0, 10.0. Print the total number of Newton iterations. Include a plot of #Newton iterations against the values of β in your report.

1.4 Container environment in Docker

Set up a Docker Container that installs an image with Ubuntu and any necessary dependencies (e.g., external libraries).

1. Write a Dockerfile and store it in your Gitlab repository.
2. Push the Container to the GitLab container registry of your repository.

1.5 CI/CD with Git

Build a CI/CD² pipeline to do the following:

1. Use the image from the container registry.
2. Build the code using CMake.
3. Test your parameter variation bash script output (the start files) against a reference solution that you created manually.
4. Test your solver with the initial and boundary conditions specified below. It is sufficient to ensure it is running successfully.

²continuous integration and continuous delivery/deployment

1.6 README.md

Write a README file that explains how to use your code to the user.

2 Scientific and algorithmic description

To solve partial differential equations (PDE), approximate numerical methods are required such as finite difference methods (FDM) which we are using in this project. The following section provides a description of the transient heat equation in two dimensions as one example of such a PDE and the solution approach via FDM followed by a description of the algorithmic structure that your implementations should follow.

2.1 Problem description

The two-dimensional transient heat equation with a non-linear source term is expressed as

$$\frac{\partial u(\mathbf{x}, t)}{\partial t} = \alpha \left(\frac{\partial^2 u(\mathbf{x}, t)}{\partial x^2} + \frac{\partial^2 u(\mathbf{x}, t)}{\partial y^2} \right) + \dot{q}(\mathbf{x}, u, t) \quad (1)$$

where u is the temperature, α is the thermal diffusivity and $\dot{q}$ is a source term. In our case, the source term is the non-linear and time dependent function

$$\begin{aligned} \dot{q}(\mathbf{x}, u, t) = & -2\pi \sin^2(\pi x) \cdot \sin^2(\pi y) \cdot \sin(\pi \gamma t) \cdot \cos(\pi \gamma t) - 2\alpha\pi [\cos^2(\pi x) - \sin^2(\pi x)] \\ & \cdot \sin^2(\pi y) \cdot \cos^2(\pi \gamma t) - 2\alpha\pi [\cos^2(\pi y) - \sin^2(\pi y)] \cdot \sin^2(\pi x) \cdot \cos^2(\pi \gamma t) - \beta \cdot u^4. \end{aligned} \quad (2)$$

The function represents an oscillating source term with additional heat loss due to radiation. Here, β is a coefficient controlling the effect of radiation and γ is a coefficient that determines the frequency of the source term. Imposing the Dirichlet boundary conditions for $u(\mathbf{x}, t)$: $u(x=0, y, t) = u(x=1, y, t) = u(x, y=0, t) = u(x, y=1, t) = 0$ and the initial conditions

$$u(\mathbf{x}, t=0) = \sin^2(\pi x) \cdot \sin^2(\pi y) \quad (3)$$

in the spatial and temporal domain $x, y \in [0, 1]$, $t \in [0, 1]$, the problem has the analytic solution for $\beta = 0$

$$u(\mathbf{x}, t) = \sin^2(\pi x) \cdot \sin^2(\pi y) \cdot \cos^2(\pi \gamma t). \quad (4)$$

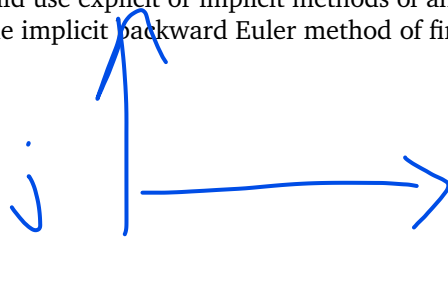
2.2 Spatial and temporal discretization

In the finite difference method, continuous derivatives are approximated with finite differences at discrete grid points. Adopting a central differencing scheme for the spatial derivatives leaves us with the semi-discretized equation

$$\frac{\partial u_{i,j}}{\partial t} \approx \alpha \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{(\Delta x^2)} + \alpha \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{(\Delta y^2)} + \dot{q}_{i,j} \quad (5)$$

where Δx^2 and Δy^2 is the grid spacing in the x and y dimensions, respectively. Therefore, the solution of the temperature at the grid point i,j depends on five values - the value at i,j itself and the values north ($j+1$), south ($j-1$), east ($i+1$) and west ($i-1$) of it.

After discretization of the spatial derivatives, only algebraic equations are left on the right hand side of eq. 5. This semi-discrete formulation represents an ordinary differential equation (ODE) and can be solved using any method for solving ODEs. One could use explicit or implicit methods of any order. However, due to simplicity, in this project we will restrict ourselves to the implicit backward Euler method of first order.



How do I make a 2d Function

How do I take time changes and 2d directional steering along with it? 4

Now we need to treat the time derivative. Consider the ODE

$$\frac{dy}{dt} = f(t, y). \quad (6)$$

The backward Euler method computes the approximation using

$$y^{k+1} = y^k + \Delta t f(t^{k+1}, y^{k+1}) \quad (7)$$

where k and $k + 1$ represent the old and new time step, respectively. Applying this method to eq. 5, leads to

$$u_{i,j}^{k+1} = u_{i,j}^k + \Delta t \alpha \frac{u_{i+1,j}^{k+1} - 2u_{i,j}^{k+1} + u_{i-1,j}^{k+1}}{(\Delta x^2)} + \Delta t \alpha \frac{u_{i,j+1}^{k+1} - 2u_{i,j}^{k+1} + u_{i,j-1}^{k+1}}{(\Delta y^2)} + \Delta t \dot{q}_{i,j}^{k+1}. \quad (8)$$

Note that eq. 8 is implicit as u^{k+1} appears on both sides of the equation. Two different solution methods can be applied depending on whether the source term is linear (independent of u) or non-linear (dependent on u).

System of linear equations For $\beta = 0$, the source term is linear and we can solve for u^{k+1} by re-arranging the equation into a system of linear equations of the form

$$\mathbf{A}u^{k+1} = u^k + \Delta t \dot{q}_i^{k+1} j \quad (9)$$

and solving that system. The matrix $\mathbf{A}$ is assembled as follows:

$$A_{ij} = \begin{cases} 1 + 4C & \text{for } j = i \\ -C & \text{for } j = i - 1 \text{ and } j = i + 1 \\ -C & \text{for } i = j - n \text{ and } i = j + n \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

System of non-linear equations For $\beta \neq 0$ the equation is non-linear and thus we need a method for solving non-linear equations. We will use a Newton method (more precisely an inexact Newton method). The Newton method linearizes a non-linear equation around a starting point and iteratively solves for it's roots. For a system of equations $g(\mathbf{x}) = 0$, the solution of at the next iteration $k + 1$ is obtained by

$$\mathbf{x}^{k+1} = \mathbf{x}^k - (\mathbf{J}(\mathbf{x}^k))^{-1} g(\mathbf{x}^k), \quad (11)$$

but is more conveniently solved instead by

$$\mathbf{J}(\mathbf{x}^k) \Delta \mathbf{x}^k = -g(\mathbf{x}^k) \quad (12)$$

with the solution increment $\Delta \mathbf{x}^k = \mathbf{x}^{k+1} - \mathbf{x}^k$ and the Jacobian $J_{ij}(x) = \frac{\partial g_i(x)}{\partial x_j}$. The Jacobian matrix represents the partial derivatives of the ODE system's RHS function to the solution variables. To apply the Newton method to our problem, we express eq. 8 in residual form

$$g(\mathbf{u}) = u_{i,j}^{k+1} - \Delta t \alpha \frac{u_{i+1,j}^{k+1} - 2u_{i,j}^{k+1} + u_{i-1,j}^{k+1}}{(\Delta x^2)} - \Delta t \alpha \frac{u_{i,j+1}^{k+1} - 2u_{i,j}^{k+1} + u_{i,j-1}^{k+1}}{(\Delta y^2)} - \Delta t \dot{q}_{i,j}^{k+1} - u_{i,j}^k = 0 \quad (13)$$

which leads to the Jacobian

$$J_{ij} = \begin{cases} 1 + 4C + 4\Delta t \beta \cdot u^3 & \text{for } j = i \\ -C & \text{for } j = i - 1 \text{ and } j = i + 1 \\ -C & \text{for } i = j - n \text{ and } i = j + n \\ 0 & \text{otherwise} \end{cases} \quad (14)$$

with $C = \Delta t \alpha / (\Delta x^2) = \Delta t \alpha / (\Delta y^2)$ and n the number of grid points in every direction. Note, the Jacobian is a band matrix with dimensions $N \times N$ (with $N = n \times n$), but only five non-zero diagonals.

2.3 Algorithmic solution

In this section, the algorithms for the main program, the Newton method, and the assembly of the Jacobian matrix are given:

Algorithm 1 Main program

```
1: Create data structures
2: Set initial conditions
3: for  $timestep\ n = 1, 2, \dots, N_{steps}$  do
4:   Update the time  $t = n * dt$ 
5:   Copy old solution of  $u$  to current solution
6:   Compute the Jacobian matrix
7:   Perform Newton iteration
8:   Copy new solution of  $u$  to old solution
9:   Optional: Calculate error norm based on analytical solution
10:  Optional: Write to file
11:  Write information to stdout
12: end for
```

Algorithm 2 Newton method

```
1: for  $iteration\ i = 1, 2, \dots, N_{max\ iterations}$  do
2:   Compute  $g(u^k)$  and enforce boundary conditions
3:   Solve the linear system  $J(u^k)\Delta u = -g(u^k)$ 
4:   Update  $u^{k+1} = u^k + \Delta u$ 
5:   if  $|\Delta u| \leq tolerance$  then
6:     Terminate loop
7:   else
8:     Continue
9:   end if
10: end for
```

3 Final report

3.1 Content

In addition to your code, write a project report. This report should be split into three parts. Part one—*Technical background*—should summarize the most important information about the tools that you needed for the project (commands, syntax,...). Please provide sources in a bibliography (the sources can also be websites, videos, lecture slides).

Use part two—*Project execution*—to reflect on what challenges arose and how you solved them. What was difficult, what was easy? How would you approach certain tasks differently in the future?

Run your solver with different parameters, e.g., different timesteps, with and without the radiation term in the source term, and a parameter of your choice, and discuss the results. Provide sources where necessary.

Show results in part three—*Results*. Discuss the results of the parameter variations.

3.2 Form

The report should be no more than 4000 words (excluding bibliography—ca. 10 pages) and be submitted as PDF. You are free to use any template. However, it should allow for code blocks and bibliography.

Algorithm 3 Jacobian

```
1: for index  $i = 1, 2, \dots, n = N_x = N_y$  do
2:   for index  $j = 1, 2, \dots, n$  do
3:      $k = i * n + j$ 
4:      $J_{k,k} = 1 + 4C + 4\beta \cdot u^3$ 
5:     if  $j < n - 1$  then
6:        $J_{k,k+1} = -C$ 
7:     else if  $j > 0$  then
8:        $J_{k,k-1} = -C$ 
9:     else if  $i < N - 1$  then
10:       $J_{k,k+n} = -C$ 
11:    else if  $i > 0$  then
12:       $J_{k,k-n} = -C$ 
13:    end if
14:  end for
15: end for
```
